
FEB 2 - PROMPT ENGINEERING

CHATBOTS

- **Let us familiarize ourselves with three chatbots:**
 - ChatGPT
 - Claude
 - Gemini (Google)
- Perplexity (allows one to select the model through an interface; good at not hallucinating; may be good for coding?)
- Make an account, explore settings (I'll demonstrate).

TO DO

- Let's take some time doing the following:
- 1) Brainstorm about what kind of things we like to do with LLMs.
- 2) Decide on a couple of tasks (perhaps one personal, one professional).
- 3) **Pick one engine and one prompt:** change settings, and seeing what happens; change prompt and see what happens.



PRINCIPLES OF PROMPT ENGINEERING

I asked ChatGPT, Claude, and Gemini the same vague question: can you recommend some general rules for prompt engineering.

GEMINI

Prompt engineering is less about "coding" and more about being a precise project manager for a very literal-minded assistant. If you give vague instructions, you'll get vague results. Here are the essential rules for getting the most out of an AI.

1. The Context Sandwich (Role, Task, Constraint)

Don't just ask for a "summary." Define the perspective and the boundaries. A well-structured prompt usually follows this flow:

- Persona:** "Act as a senior marketing consultant..."
- Task:** "...critique this landing page copy..."
- Context:** "...for a high-end eco-friendly skincare brand targeting Gen Z."
- Constraint:** "...keep the tone punchy and avoid corporate jargon."

2. Be Specific with Output Format

If you don't specify how you want the data, the AI will default to a block of text. Be explicit about the structure:

- Tables:** "Present the pros and cons in a table."
- Lists:** "Give me 5 bullet points, each no longer than 15 words."
- Code/Markdown:** "Format the output as a JSON object" or "Use H3 headers for each section."

3. Use "Few-Shot" Prompting (Examples)

Providing examples is the single most effective way to improve output quality. Instead of describing a style, **show it**.

"Categorize these emails based on sentiment."

Example 1: 'I hate this product!' -> Sentiment: Angry

Example 2: 'It's okay, I guess.' -> Sentiment: Neutral

Input: 'This is the best thing I've ever bought!'"

4. Let the AI "Think" (Chain of Thought)

For complex logic or math, ask the AI to **"think step-by-step"** before giving the final answer.

This forces the model to process the logic linearly, which significantly reduces "hallucinations" (making stuff up).

5. Negative Constraints (The "Anti-Prompt")

Telling the AI what *not* to do is just as important as telling it what to do.

- "Do not use the word 'delve' or 'tapestry'."** "Avoid mentioning any competitors." "Don't provide an intro or outro; just give me the raw data."

GEMINI – SUMMARY TABLR

Rule

Persona

Delimiters

Iteration

Temperature

Why it Works

Sets the tone and depth of knowledge.

Using "" or ### helps the AI distinguish between instructions and the text you want it to process.

Treat your first prompt as a draft. Refine based on the first "bad" answer.

(In settings) Lower for facts/logic, higher for creative writing.

CLAUDE

- I'd be happy to share some effective prompt engineering practices! Here are the key principles that tend to work well:
- **Be clear and specific.** Rather than vague requests, give Claude enough detail to understand what you're looking for. Instead of "write about climate change," try "write a 300-word explanation of how carbon dioxide traps heat in the atmosphere, aimed at high school students."
- **Use examples.** Showing Claude examples of what you want—whether it's a writing style, format, or type of answer—is often more effective than just describing it. If you want a certain tone or structure, providing one or two examples helps Claude understand your expectations.
- **Break down complex tasks.** If you're asking Claude to do something multifaceted, it often helps to ask it to work through the problem step-by-step or to break it into stages. You can also ask Claude to explain its reasoning before giving you a final answer.
- **Be direct about format.** If you want output in a specific format—like a table, code snippet, JSON, or outlined points—just say so. You can also use XML tags to structure requests (for example, asking Claude to put analysis in <analysis> tags).
- **Specify length or detail level.** Mention if you want something concise or comprehensive, or give a word/page count if that's relevant.
- **Iterate and refine.** If Claude's first response isn't quite right, you can build on it—ask follow-up questions, request adjustments, or clarify what wasn't working.
- For more comprehensive guidance on prompting, Anthropic has detailed documentation at <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview> that goes deeper into these and other techniques.

CHATGPT

- Below is a **precise, high-signal set of prompt-engineering principles** that work reliably for scientific, technical, and creative workflows alike. I've tailored them to your typical use cases (ML for climate/ocean, astrophysics, curriculum design, proposal writing, etc.).

General Rules for Prompt Engineering

1. State the task explicitly

LLMs optimize for *what you ask*, not what you intend.

Define the task category up front:

- “Summarize...”
- “Rewrite for a scientific audience...”
- “Produce a 2-slide explanation...”
- “Generate Python using xarray and cartopy...”

Rule: Name the task before giving details.

2. Specify the desired output structure

LLMs perform best when the expected format is fixed:

- bullet list
- numbered steps
- JSON
- 500-word abstract
- 5-slide deck outline
- 2-column table
- LaTeX block

Rule: Structure → quality.

3. Provide domain context

For expert tasks—your norm—give minimal but essential context:

- Dataset characteristics (SOCAT spatiotemporal density, mixed-resolution covariates)
- Scientific framing (mCDR detection thresholds, SED-fitting degeneracies)
- Audience (4th-grade, NSF panelists, Grad Center course)

Rule: Give the model the same priors an expert human would need.

4. Constrain the style, tone, and register

Control registers explicitly:

- “Write in a concise, technical tone as in a Methods section.”
- “Explain as if teaching to MS/PhD students.”
- “Use Montale’s compressed, imagistic style.”

Rule: If you don’t constrain tone, the model picks one for you.

CHATGPT

5. Give positive and negative examples

Few-shot prompting is extremely powerful:

Good example:

"Here is a strong abstract in this style → now rewrite the following text similarly."

Counterexamples:

"Avoid generic statements such as X and avoid anthropomorphizing models."

Rule: Examples == grounding.

7. Break complex tasks into subtasks

Many tasks work better as *chained prompts*:

- Extract key claims.
- Reframe for climate-ML audience.
- Compress to 150 words.
- Convert to slide bullet points.

Rule: Decompose like you would for a student or RA.

9. Ask for the model's reasoning checks

Useful for scientific correctness:

- "List possible failure modes of your answer."*
- "Add uncertainty quantification: what are the assumptions?"*
- "Flag any statements that may require verification."*

Rule: You get higher epistemic hygiene when you ask for it.

6. Set boundaries (constraints)

Constraints prevent drift:

- maximum length
- explicitly exclude jargon or include it
- "do not invent bibliography entries"*
- "use only equations found below"*
- "no moralizing or general-purpose advice"*

Rule: Constraints focus the search space.

8. Use metadata when helpful

Tagging improves precision:

Example:

[Audience: NSF reviewers][Format: 300-word project description][Focus: broader impacts of ML for ocean carbon

Rule: Metadata cues reliably shape the output manifold.

10. Iterate with targeted refinement

LLMs respond extremely well to surgical constraints:

- "Shorten but keep technical density."*
- "Add 2 references with DOIs."*
- "Add one concrete oceanographic example (e.g., SOCAT sparsity)."*

Rule: Prompting is iterative, not one-shot.

CHATGPT

Pro-level patterns (highly effective in your research pipeline)

A. Role prompting (but minimal)

Works well when combined with constraints:

- *“Act as a climate-ML researcher with expertise in mixed-resolution spatiotemporal fusion.”*

Avoid overlong role-play—clarity > theatrics.

B. Reflection prompts

Ask the model to optimize its own output:

- *“Before answering, list the 3 key considerations this task requires.”*
- *“Regenerate the same text with tighter logical flow.”*

C. Self-consistency sampling

Use:

“Provide 3 alternative phrasings.”

Then select the best. Excellent for abstracts, slide captions, and justification text.

D. Constraint-first prompting

State all constraints first, then the text:

Constraints: • Max 250 words • No generic climate statements • Must cite two peer-reviewed ocean-carbon papers • Write for intelligent non-specialists Task: Rewrite the following text...

Highest reliability for grant writing.

E. Verification prompts

For your ML/ocean workflows:

- *“Check whether any equation assumptions contradict standard pCO_2/TCO_2 parameterizations.”*
- *“Verify dimensional consistency.”*

CHATGPT: NEW PROMPT

I would like a one-pager summarizing the core prompt-engineering principles for my LLMs for Science course, which includes masters and PhD students in physics, data visualization, and Earth and environmental science.

LLMs for Science — Core Prompt-Engineering Principles (One-Pager)

1. Define the Task with Surgical Precision

LLMs optimize for *explicit* intent.

State the task category up front:

- “Summarize the Methods section...”
- “Generate Python using xarray for netCDF analysis...”
- “Rewrite for a scientific but non-expert audience...”

Why it matters for science:

Ambiguous tasks lead to hallucinations or generic text.

3. Provide Domain Context (Just Enough)

Give the model the minimal priors an expert human would need:

- Dataset characteristics (SOCAT resolution, ESM grids)
- Scientific framing (uncertainty, physical constraints)
- Audience (PI, NSF panel, cross-disciplinary students)

Result:

Sharper reasoning, fewer incorrect assumptions.

5. Use Examples (Few-Shot Prompting)

Show 1–2 prototypes of the style or format you want.

Include negative examples when needed:

- “Do not use generic statements about climate.”

Effect:

Anchors the model and reduces variance.

9. Metadata Prompting

Add metadata tags at the top of complex prompts:

[Audience: PhD students in Earth & Environmental Science][Format: 2-slide summary][Focus: uncertainty in mixed-resolution covariates]

LLMs use this reliably to shape style and depth.

4. Constrain Style and Register

Control tone explicitly:

- technical, didactic, concise, expository, review-style*
- “Use the tone of a Methods section.”
- “Avoid anthropomorphizing models.”

In Earth/physics contexts:

Prevents over-confident narrative text.

7. Decompose Complex Tasks

Break multistep work into sequential subtasks:

Extract key claims

Identify assumptions

Rewrite for target audience

Produce LaTeX equations or code

Why:

Modularity mirrors scientific reasoning and improves fidelity.

2. Specify Output Structure

Explicit structure → reproducible outputs.

- Bullet list, numbered steps
- JSON / YAML / tables
- 150-word abstract
- 2-slide explanation
- Equation blocks in LaTeX

In scientific workflows:

Requiring structure prevents drift and ensures the LLM stays compatible with co

6. State Hard Constraints

Examples:

- max 200 words
- cite 2 peer-reviewed papers
- “Do not invent data or references.”
- “Use only variables defined below.”

Scientific benefit:

Avoids hallucinations and uncontrolled extrapolation.

8. Use Reflection and Verification Prompts

Ask the model to critique its own output:

- “List possible failure modes of your answer.”
- “Check dimensional consistency.”
- “Flag statements requiring external verification.”

Particularly powerful in physics and EES,

where physical validity is essential.

10. Iterate with Micro-Refinements

After the initial output:

- “Shorten by 20% without losing technical density.”
- “Add an example using SOCAT or GLODAP.”
- “Regenerate with clearer logic.”

Iterative prompting is often more effective than a single long prompt.

TO DO (AGAIN)

- Let's take some time doing the following:
- 1) Brainstorm about what kind of things we like to do with LLMs.
- 2) Decide on a couple of tasks (perhaps one personal, one professional).
- 3) **Pick one engine and one prompt:** change settings, and seeing what happens; change prompt and see what happens.
- 4) If you are happy about your prompt and have time, you can try the same prompt with a different chatbot.
- 5) Report in our shared document (called Results from Prompt...) the result of your experiment:
 - Initial prompt and output
 - Effect of change in settings
 - Effect of change in prompt
 - (if you got there), difference in output across chatbots
 - 1-2 sentences as your conclusion

FOR WEDNESDAY

- Read
- <https://www.technologyreview.com/2025/05/20/1116327/ai-energy-usage-climate-footprint-big-tech/>
- <https://www.patreon.com/posts/when-ai-hurts-it-148885841>
- and also watch
- https://www.youtube.com/watch?v=Bl-vPf_IAoA
- Wednesday: Matt Lease seminar 12pm - 12.30pm
- <https://cosmicai.org/events/talk-2-spring-2026-cosmicai-hybrid-seminar>